Introduction to Software Systems
COMP 206

Instructors: Jacob Errington & Max Kopinsky
School of Computer Science
McGill University

Lesson 4: standard streams, redirection, and pipes

Outline

- Standard streams.
- Redirection: reading and writing files with bash.
- Pipes: combining multiple programs together.

Standard streams:

stdin, **stdout**, and **stderr**

Program vs process

Program. Something you can run on a computer, e.g. a file containing Python code.

Process. A running instance of a program.

Program vs process

Program. Something you can run on a computer, e.g. a file containing Python code.

Process. A running instance of a program.

OOP analogy: a program is like a class, a process is like an instance of that class.

Program vs process

Program. Something you can run on a computer, e.g. a file containing Python code.

Process. A running instance of a program.

OOP analogy: a program is like a class, a process is like an instance of that class.

- Processes can open files for reading and writing.
- Each opened file is associated (by the OS) to a numeric ID called a file descriptor.

The standard streams

- Be default, each process has three (special) file descriptors.

The standard streams

- Be default, each process has three (special) file descriptors.
  - FD 0 is called standard input, stdin.
  - FD 1 is called standard output, stdout.
  - FD 2 is called standard error, stderr.
- These are not usually "files" in the traditional sense!

  e.g. bash has its stdin, stdout, and stderr attached to the terminal (running on your computer).

  Ex. `cat instructions.md` → the output is displayed on your terminal.

The standard streams

- Be default, each process has three (special) file descriptors.
  - FD 0 is called standard input, stdin.
  - FD 1 is called standard output, stdout.
  - FD 2 is called standard error, stderr.
- These are not usually "files" in the traditional sense!

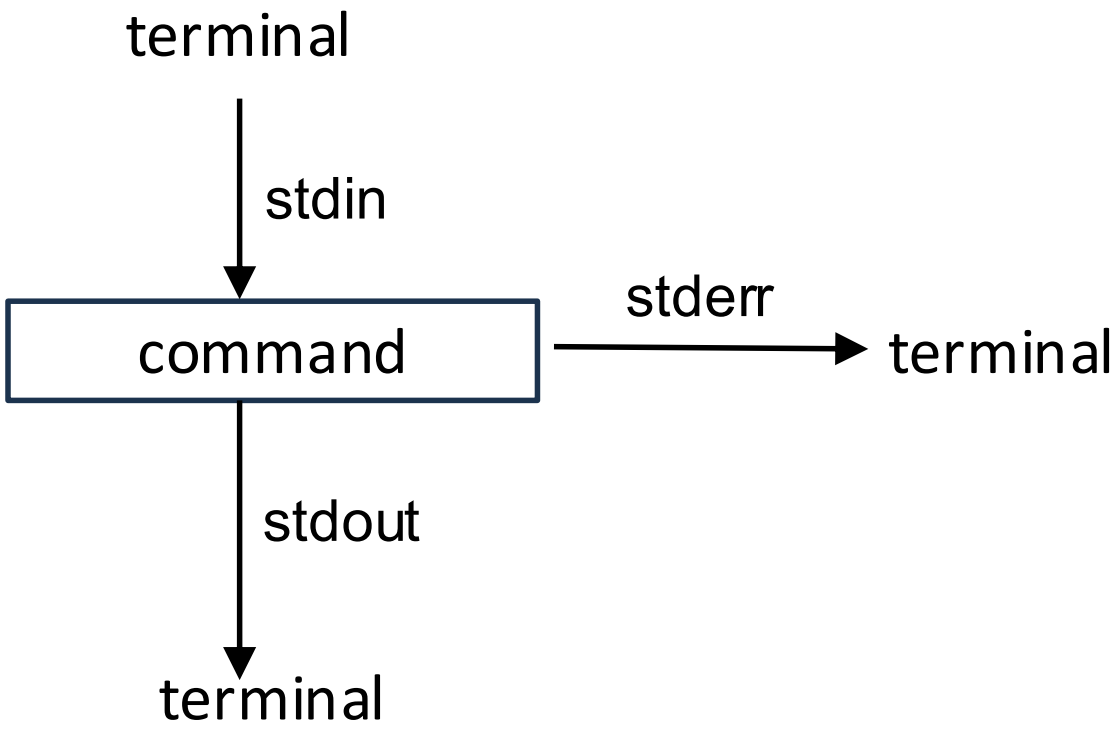
  e.g. bash has its stdin, stdout, and stderr attached to the terminal (running on your computer).

  Ex. `cat instructions.md` → the output is displayed on your terminal.
- Using the standard streams in Python: input(), print() and sys.stderr.write().

Redirection: Controlling the Standard Streams

- By default, standard streams (stdin, stdout, stderr) are attached to your terminal.

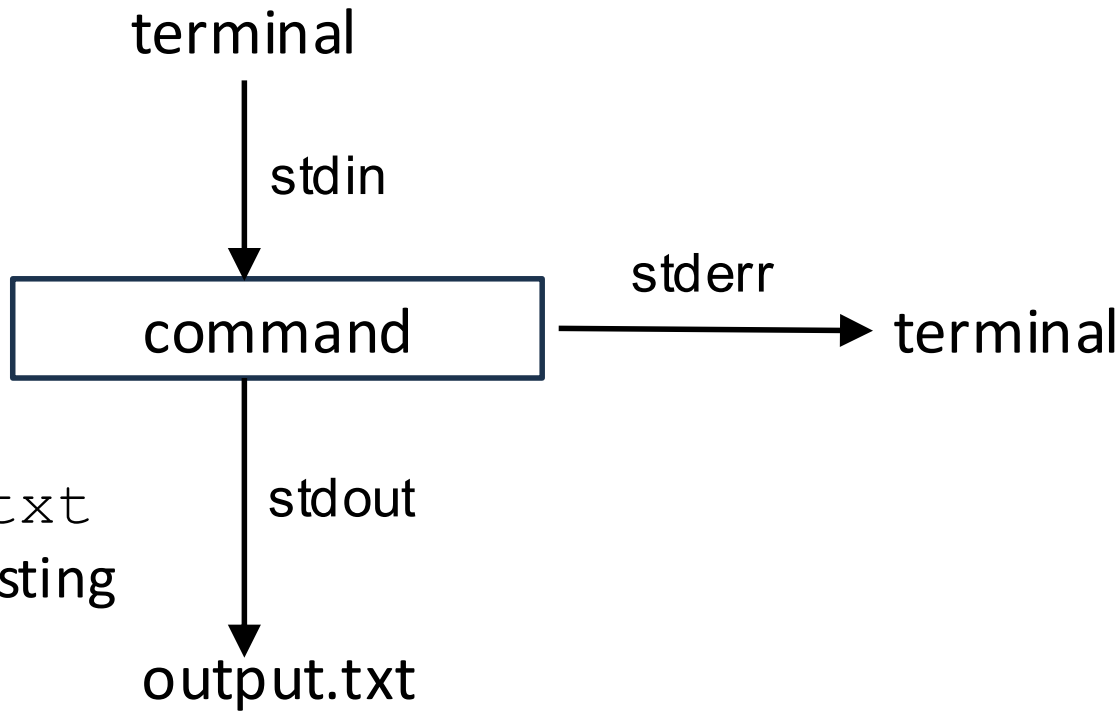
- `$ command`



Redirection: Controlling the Standard Streams

- However, there is syntax in bash – called redirection – to instead take stdin from a file, or to send stdout/err to a file.

- `$ command > output.txt`

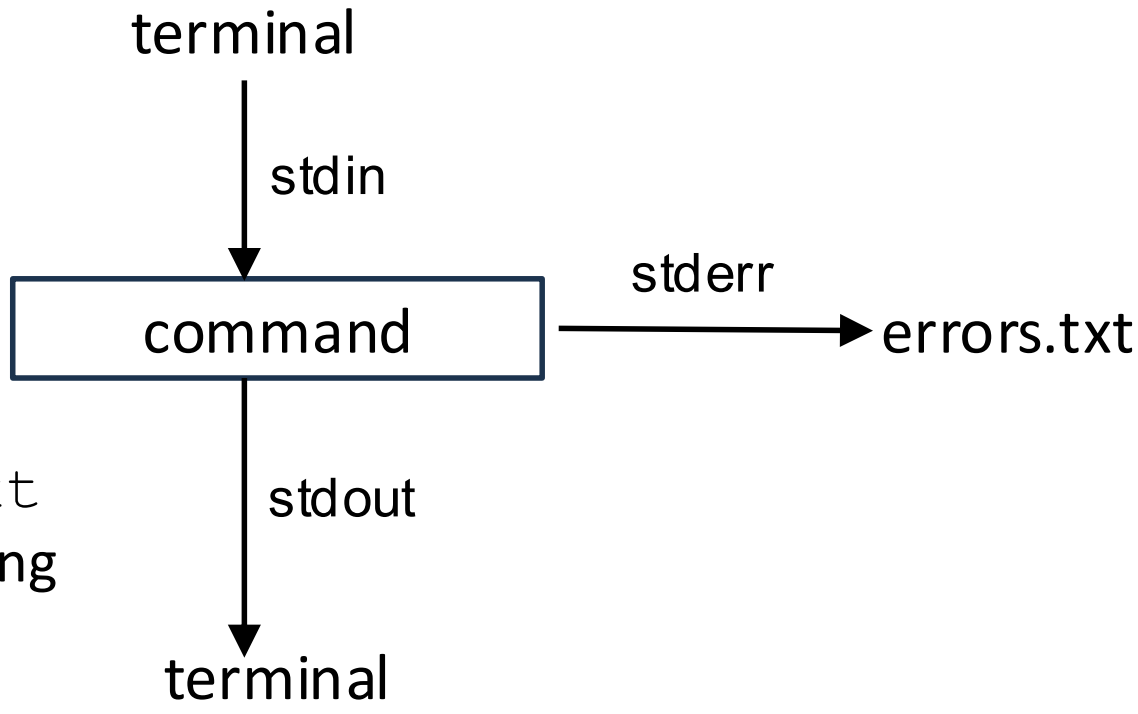


`$ command >> output.txt`
redirect output, append to existing file

Redirection: Controlling the Standard Streams

- However, there is syntax in bash – called redirection – to instead take stdin from a file, or to send stdout/err to a file.

- `$ command 2> errors.txt`



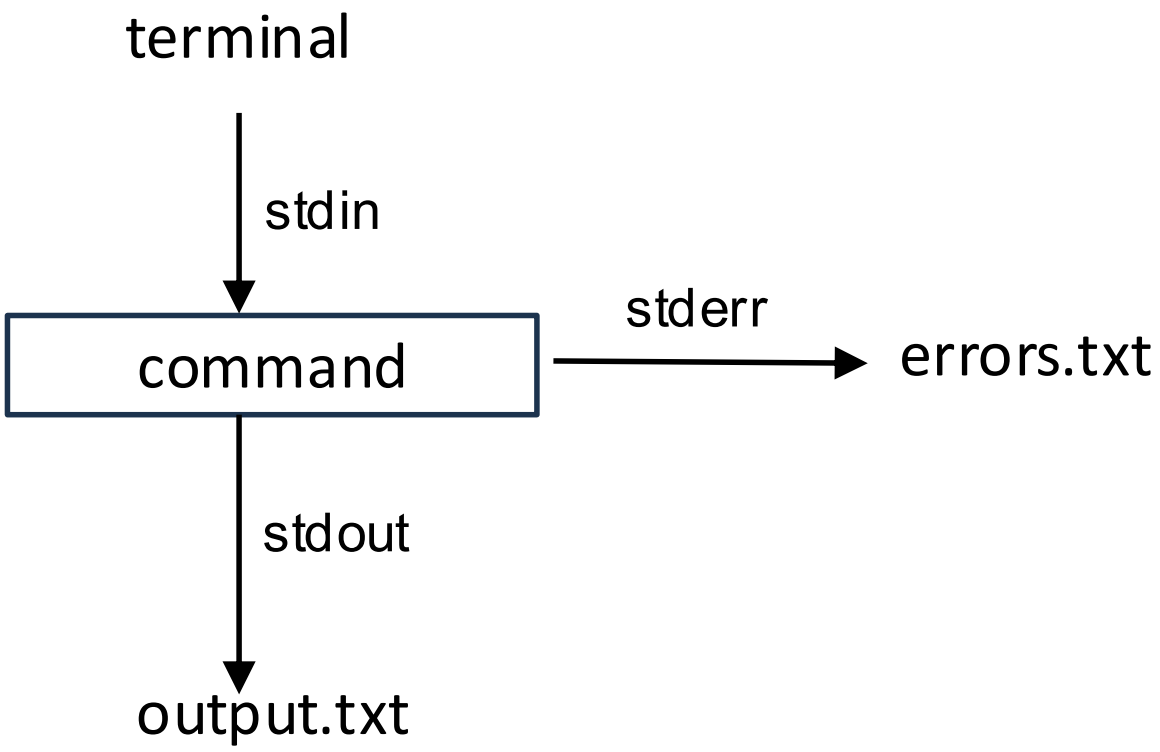
$ `command 2>> errors.txt`
redirect output, append to existing file

Redirection: Controlling the Standard Streams

- However, there is syntax in bash – called redirection – to instead take stdin from a file, or to send stdout/err to a file.

- `$ command 2> errors.txt > output.txt`

  redirect both stdout and stderr to a file

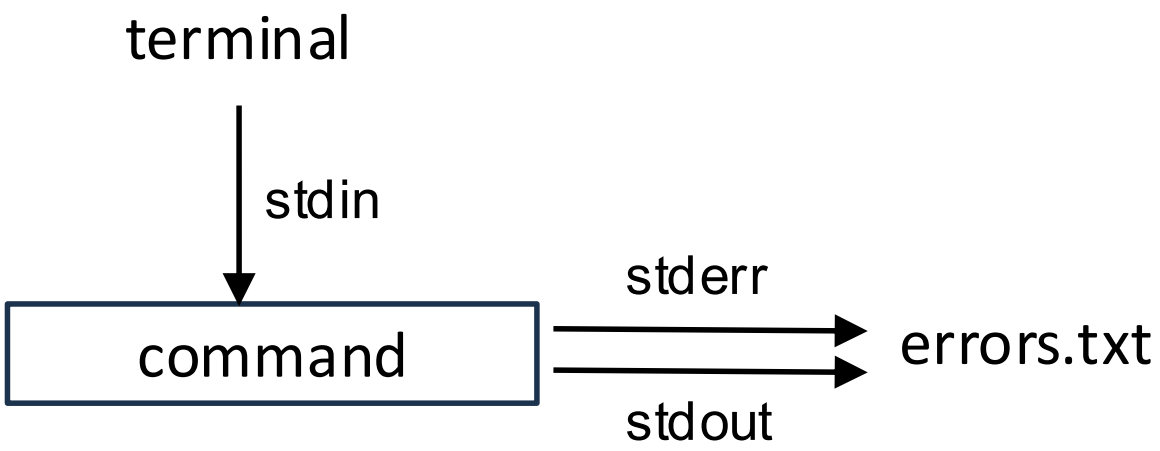


Redirection: Controlling the Standard Streams

- However, there is syntax in bash – called redirection – to instead take stdin from a file, or to send stdout/err to a file.

- `$ command 2> errors.txt >&2`

  Redirections are applied in order.

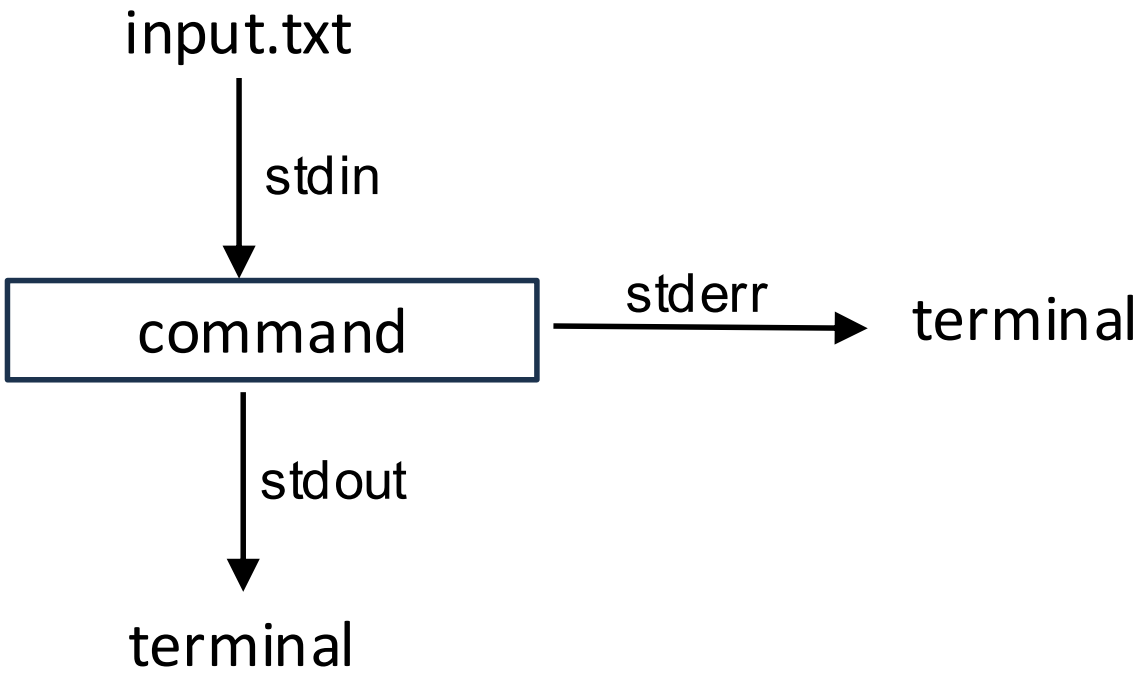


Redirection: Controlling the Standard Streams

- However, there is syntax in bash – called redirection – to instead take stdin from a file, or to send stdout/err to a file.

- `$ command < input.txt`

  take stdin from a file



Redirection: Controlling the Standard Streams

- Examples:
  - `$ ls -al > directorylist.txt`
  - `$ cat ~/comp-206/homework/hw1/instructions.md >> directorylist.txt`
  - `$ less directorylist.txt`

Piping: Sending stdout of one program to stdin of another

- Output from one program (stdout) sent as input to another program (stdin)
- Symbol: |
- Ex: `$ cat directorylist.txt | less`

  `$ ls -la | less` will present a paginate list of files.

  `$ ls -la | head` will present only the first 10 files.

  `$ ls -la | tail` will present only the last 10 files.

Piping and redirection in the same command

- `$ cat $(ls | tail -n5) > text.out`

  concatenate the last 5 files in the current directory and write them to text.out file.

  - Notice $ () after cat is for Nested execution (command substitution)